Task 1 – GUI Design Review (6 Marks, K4)

Run each program yourself and attach your own screenshots, each linked to the requirement or test it shows. Items marked `<fill>` need your own observations or measurements.

Task 1.1 – Diagnose the design (8 deficiencies)

#	Deficiency	What the user experiences / what goes wrong
1	**No separation of interface and logic.** `FeeWindow` parses input, opens JDBC, builds SQL and sleeps, all in the listener.	Impossible to unit-test without a window and a database; any change to payment rules means editing GUI code.
2	**`e.getActionCommand() == "Pay"` compares references, not text.**	It only works by luck of string interning. Use `equals()`, or better, give each button its own listener.
3	**No validation of empty or invalid input.** `Double.parseDouble("")` throws `NumberFormatException`.	The exception is thrown on the event-dispatch thread and printed to the console. The user sees nothing happen. A negative or zero amount would be inserted.
4	**Database work and `Thread.sleep(3000)` on the EDT.**	The window freezes for three seconds: no repainting, buttons unresponsive, title bar shows "Not responding". Use `SwingWorker`.
5	**No feedback in the GUI.** Success is `System.out.println("Paid")`.	The user cannot tell whether the payment worked, failed or is still running.
6	**The Receipt button has no handler.** Its click reaches the `else` branch.	Clicking Receipt **clears** both fields, which is wrong and loses the user's data.
7	**SQL built by string**	SQL injection: a register

#	Deficiency	What the user experiences / what goes wrong
	concatenation.	number such as `x','0');DROP TABLE fees;--` can alter the database. Use `PreparedStatement`.
8	**Connection never closed; window cannot be closed; checked exceptions unhandled.** `SQLException` and `InterruptedException` are not caught. `actionPerformed` declares no `throws`, so the code does not even compile.	Leaked connections exhaust the server. There is no `WindowListener`, so the close button does nothing.
9	(extra) Buttons are never added to the frame and no listener is registered with `addActionListener`.	Even if it compiled, nothing would be shown and nothing would respond.

Task 1.2 – Repair the design

Class structure

```
FeeWindow (JFrame, view)  ──uses──>  FeeService (validation + payment
rule)
                                          | depends on interface
                                          ▼
                                   FeeRepository  <──
JdbcFeeRepository (PreparedStatement, try-with-resources)
```

`FeeWindow` only reads text fields, calls the service and shows the result. `FeeService` knows nothing about Swing. `JdbcFeeRepository` is the only class that knows SQL.

FeeService.java

```
// Business logic: no GUI classes here, so it can be unit-tested.
public class FeeService {
    public interface FeeRepository { void save(String regNo, double
amount) throws Exception; }

    private final FeeRepository repo;
    public FeeService(FeeRepository repo) { this.repo = repo; }
```

```
    /** Returns null if valid, otherwise a user-readable error. */
    public String validate(String regNo, String amountText) {
        if (regNo == null || regNo.isBlank()) return "Register number
is required";
        if (amountText == null || amountText.isBlank()) return "Amount
is required";
        try {
            double a = Double.parseDouble(amountText.trim());
            if (a <= 0) return "Amount must be positive";
        } catch (NumberFormatException e) { return "Amount must be a
number"; }
        return null;
    }

    public void pay(String regNo, String amountText) throws Exception
{
        String err = validate(regNo, amountText);
        if (err != null) throw new IllegalArgumentException(err);
        repo.save(regNo.trim(),
Double.parseDouble(amountText.trim()));
    }
}
```

JdbcFeeRepository.java

```
import java.sql.*;

public class JdbcFeeRepository implements FeeService.FeeRepository {
    private final String url, user, pwd;
    public JdbcFeeRepository(String url, String user, String pwd) {
this.url = url; this.user = user; this.pwd = pwd; }

    @Override public void save(String regNo, double amount) throws
SQLException {
        String sql = "INSERT INTO fees(reg_no, amount) VALUES (?, ?)";
// no string concatenation
        try (Connection c = DriverManager.getConnection(url, user,
pwd);   // always closed
             PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, regNo);
            ps.setDouble(2, amount);
            ps.executeUpdate();
        }
    }
}
```

FeeWindow.java (one listener per button, DB work in a SwingWorker)

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
```

```
/** View only: collects input, shows results. */
public class FeeWindow extends JFrame {
    private final JTextField regNo = new JTextField(12), amount = new
JTextField(8);
    private final JLabel status = new JLabel(" ");
    private final JButton pay = new JButton("Pay"), clear = new
JButton("Clear"), receipt = new JButton("Receipt");
    private final FeeService service;
    private String lastReceipt = null;

    public FeeWindow(FeeService service) {
        super("Fee Payment");
        this.service = service;
        setLayout(new FlowLayout());
        add(new JLabel("Reg No")); add(regNo); add(new
JLabel("Amount")); add(amount);
        add(pay); add(clear); add(receipt); add(status);

        pay.addActionListener(e -> doPay());                        //
one listener per button
        clear.addActionListener(e -> { regNo.setText("");
amount.setText(""); status.setText(" "); });
        receipt.addActionListener(e ->
JOptionPane.showMessageDialog(this,
                lastReceipt == null ? "No payment yet" :
lastReceipt));

        setDefaultCloseOperation(DISPOSE_ON_CLOSE);
        pack();
    }

    private void doPay() {
        String err = service.validate(regNo.getText(),
amount.getText());
        if (err != null) { status.setText(err); return; }
        pay.setEnabled(false);
        status.setText("Processing...");
        new SwingWorker<Void, Void>() {                             //
DB work off the EDT
            @Override protected Void doInBackground() throws Exception
{
                service.pay(regNo.getText(), amount.getText());
                return null;
            }
            @Override protected void done() {
                pay.setEnabled(true);
                try {
                    get();
                    lastReceipt = "Paid " + amount.getText() + " for "
```

```
+ regNo.getText();
                    status.setText("Payment successful");
                } catch (Exception ex) {
                    status.setText("Payment failed: " + (ex.getCause()
!= null ? ex.getCause().getMessage() : ex.getMessage()));
                }
            }
        }.execute();
    }

    public static void main(String[] a) {
        FeeService svc = new FeeService(new
JdbcFeeRepository("jdbc:mysql://localhost/college", "user", "pwd"));
        SwingUtilities.invokeLater(() -> new
FeeWindow(svc).setVisible(true));
    }
}
```

Test without any GUI or database (ServiceTest.java)

```
public class ServiceTest {
    public static void main(String[] a) throws Exception {
        java.util.List<String> saved = new java.util.ArrayList<>();
        FeeService s = new FeeService((r, amt) -> saved.add(r + ":" +
amt));   // fake repo: no DB, no GUI
        System.out.println(s.validate("", "100"));
        System.out.println(s.validate("R1", "abc"));
        System.out.println(s.validate("R1", "-5"));
        System.out.println(s.validate("R1", "500"));
        s.pay("R1", "500");
        System.out.println(saved);
    }
}
```

Output:

```
Register number is required
Amount must be a number
Amount must be positive
null            <- null means valid
[R1:500.0]
```

SQL for the table: `CREATE TABLE fees(reg_no VARCHAR(20), amount DECIMAL(10,2));`

Task 1.3 – Design defence (under 100 words)

The window now only collects input and displays results, while `FeeService` holds the rules and `JdbcFeeRepository` holds SQL. The service can be tested with a fake repository in milliseconds, with no window or database (see `ServiceTest`). A change to validation, the database or the screen layout touches exactly one class. Each button has its own small listener,

so adding a button cannot break the others. Payments run in a `SwingWorker`, so the window stays responsive, and prepared statements remove SQL injection.

Alternative concept and complexity

One listener with an if–else chain vs. one listener object per button (Swing `Action`):

```
Action pay = new AbstractAction("Pay") { public void
actionPerformed(ActionEvent e) { doPay(); } };
payButton.setAction(pay);          // a menu item or toolbar button
can share the same Action
```

- **Time to dispatch an event, k buttons:** the if–else chain on action commands makes up to k string comparisons, so O(k), average k/2 and worst case k, each comparison costing O(length of the string). With one listener per button the source already holds its listener, so Swing calls it directly: O(1).
- **Space:** a single listener is O(1) objects. k listeners are O(k) objects, a few dozen bytes each. For a screen with a handful of buttons both are negligible, and the clarity of separate listeners is worth far more than the memory.
- **Own measurement `<fill>`:** time 1,000,000 dispatches with k = 5 and k = 50 using `System.nanoTime()`; the chain grows with k, the per-button version stays flat.